

ROADMAP DA APROVAÇÃO — ETB-PAY

Dossiê de Revisão de Banco de Dados | Turma 2G

1. Mensagem do Tech Lead

Parabéns a toda a Squad 2G! Chegamos a um marco crucial do nosso desenvolvimento. Como um Engenheiro de Dados focado na excelência e em conformidade estrita com concursos públicos de alto padrão (como as exigências da banca CEBRASPE), fico imensamente orgulhoso da evolução técnica de vocês. Vocês transitaram de meros construtores de scripts soltos para arquitetos de persistência maduros, capazes de modelar estruturas relacionais complexas e altamente eficientes.

Mais do que apenas salvar dados, vocês dominaram os desafios do mundo real: a extração, transformação e carga (ETL), o indispensável **Saneamento de Dados (Data Cleaning)** que purifica as informações essenciais para relatórios corporativos, e, o mais vital para o ETB-PAY, a estrita aderência ao **Compliance e Auditorias do BACEN**.

Compreender os níveis de abstração e as exigências da Álgebra Relacional (como os JOINS) eleva a capacidade técnica do grupo a um nível analítico que as maiores corporações — e os certames mais rigorosos — exigem.

Esta apostila consolida o conhecimento avançado em SGBDs para as nossas avaliações, funcionando como o seu verdadeiro guia de sobrevivência e aprovação. Devorem este material. Usem a tecnologia ao seu favor e dominem a banca.

SEÇÃO A: MERGULHO TEÓRICO EXAUSTIVO

Tópico 1: Arquitetura de SGBDs (Client-Server vs. Embedded)

O ecossistema dos Sistemas Gerenciadores de Banco de Dados é dividido arquiteturalmente de forma clara. Sistemas como **PostgreSQL, Oracle e SQL Server** adotam a arquitetura **Client-Server (Cliente-Servidor)**, onde o SGBD é executado de forma autônoma como um serviço daemon (processo em background) gerenciando requisições de rede assíncronas em uma porta específica (ex: 5432).

Em contraste absoluto, o **SQLite**, adotado no ETB-PAY, é um banco de dados **Embedded (Embutido/Serverless)**. De forma cirúrgica: **o SQLite NÃO roda como um serviço ou processo em background!** Ele é empacotado como uma biblioteca (um arquivo .dll, .so ou .jar) diretamente integrada à aplicação host. Não há tráfego de rede TCP/IP ou overhead de comunicação de soquetes; a aplicação chama o banco através de simples chamadas de função locais, manipulando um único arquivo no disco.

Cuidado com a Pegadinha de Concurso: Bancas (especialmente a CEBRASPE) adoram afirmar que o "SQLite é um serviço background robusto para controle assíncrono". Isso está tecnicamente **ERRADO**. A característica essencial do SQLite é exatamente a ausência de um processo servidor em background.

Tópico 2: Os 3 Níveis de Abstração de Dados

O projeto de Banco de Dados Relacionais segue a consagrada arquitetura ANSI/SPARC de 3 níveis, cujo objetivo é o isolamento de complexidade.

1. Nível Conceitual: Foco absoluto nas Regras de Negócio e no entendimento humano. É onde construímos o Modelo Entidade-Relacionamento (MER). Nele, desenhamos retângulos e losangos para definir que "Um Cliente Possui Várias Contas", independente de SGBD ou tipo de máquina. Representa o "O QUÊ" os dados significam.

2. Nível Lógico: É a tradução do MER para o modelo estrutural tecnológico do SGBD (ex: Relacional). Aqui as entidades viram Tabelas, estabelecemos Tipos de Dados teóricos, formalizamos a Chave Primária (PK) e a Chave Estrangeira (FK). É um desenho técnico em formato de tabelas sem ainda escrever código de máquina.

3. Nível Físico: Trata do aspecto puramente tecnológico (a máquina). É onde aplicamos os scripts DDL (Data Definition Language) com SGBD específico. Criamos índices (B-Tree, Hashing), tablespaces no HD e digitamos comandos práticos como `CREATE TABLE tb_clientes`.

Tópico 3: Álgebra Relacional e o Poder do JOIN

A Álgebra Relacional é a matemática teórica por trás do modelo relacional, estruturando a manipulação de dados em conjuntos (tabelas). Quando normalizamos as tabelas (evitando anomalias), "quebramos" os dados em múltiplos arquivos coesos. O problema: precisamos visualizar esses dados unidos em consultas gerenciais.

A solução é a **Operação de Junção (JOIN)**. O INNER JOIN, o mais comum da Álgebra Relacional, não é apenas mesclar; é atuar como uma **costura lógica exata**. Ele cruza rigorosamente a **Chave Estrangeira (FK)** da tabela dependente (filha) com a **Chave Primária (PK)** da tabela principal (pai). Só os registros que tiverem correspondência em ambas as partições do conjunto formarão a nova linha gerada em memória durante a consulta, impedindo qualquer produto cartesiano indesejado.

Tópico 4: Integridade Referencial e Restrições (CASCADE)

A **Integridade Referencial** é a principal salvaguarda de confiabilidade dentro de um banco relacional: "Nenhum registro filho pode apontar para um registro pai que não existe". Ao criar uma Foreign Key (FK), impomos essa barreira.

Se o banco de dados tenta deletar um Cliente (Pai) que ainda tem Contas (Filhas), a integridade bloqueia a operação (comportamento padrão RESTRICT). No entanto, podemos desenhar nosso DDL com o gatilho **ON DELETE CASCADE** na FK. Essa configuração permite que a base continue consistente eliminando as amarras cegas: se a PK do pai for apagada, o SGBD destruirá em cascata todas as tuplas filhas atreladas a ele automaticamente.

A falta dessa gestão cuidadosa das dependências gera o maior pesadelo dos analistas de compliance do BACEN: os **Registros Órfãos** (dados fantasma que apontam para IDs inexistentes), o que corrompe a auditoria inteira do sistema financeiro.

SEÇÃO B: ENGENHARIA REVERSA DA BANCA

Q1 (Níveis de Abstração): Conceitual vs Físico

O que a questão diz: "O Modelo Entidade-Relacionamento (MER), devido ao detalhamento de restrições das cláusulas PRIMARY KEY e FOREIGN KEY, é categorizado como modelagem de Nível Físico."

O Erro (O Quê): Classificar o MER como Nível Físico e atribuir a ele a criação de cláusulas de código de banco de dados (DDL).

A Regra (Por Quê): O MER pertence ao **Nível Conceitual**. O Conceitual ignora códigos e sintaxe (sem restrições de PK no código), focando apenas na lógica de negócios e compreensão humana através de diagramas de blocos. DDL e cláusulas de código residem no Nível Físico.

Q2 (Regras de Normalização): 1FN

O que a questão diz: "Uma tabela de clientes de banco atende à 1ª Forma Normal (1FN) mesmo quando um campo de contato armazena três números de celular separados por vírgula em uma mesma célula de coluna."

O Erro (O Quê): Afirmar que a 1FN permite atributos multivalorados na mesma tupla e campo.

A Regra (Por Quê): A Regra Áurea da 1FN é a **Atomicidade**. O dado na interseção de uma linha com coluna deve ser atômico, indivisível, contendo um, e apenas um, valor. Valores compostos (rua, bairro, cidade na mesma string) ou multivalorados (vários celulares) ferem a 1FN e devem ser extraídos para novas tabelas ou colunas.

Q3 (Regras de Normalização): 2FN

O que a questão diz: "Se a tabela possuir uma Chave Primária Composta (ID_Produto, ID_Loja), e a coluna 'Nome do Produto' depender apenas da metade da chave (ID_Produto), a tabela ainda atende a 2FN."

O Erro (O Quê): Dizer que a 2FN tolera a dependência parcial da chave primária composta.

A Regra (Por Quê): A 2FN estipula que a tabela deve estar na 1FN, e todos os atributos que não são chave devem depender da **CHAVE PRIMÁRIA COMPLETA (INTEIRA)**. Se o nome do produto depende apenas do ID do produto, ocorre "dependência parcial". A solução é extrair os dados dependentes parcialmente para uma tabela independente.

Q4 (Regras de Normalização): 3FN

O que a questão diz: "Para um banco atender à 3FN, não tem problema armazenar o 'Nome da Agência' na tabela de 'Transações Bancárias', visto que já temos o 'Código da Agência' na mesma."

O Erro (O Quê): Armazenar o "Nome da Agência" gerando o dado "caroneiro".

A Regra (Por Quê): A 3FN elimina a **dependência transitiva**. Se o "Nome da Agência" depende do "Código da Agência", que não é a Chave Primária da tabela de "Transações Bancárias", temos um atributo não-chave dependendo de outro atributo não-chave. Esse atributo caroneiro deve ser migrado para a tabela independente de Agências.

Q5 (Comandos DDL vs DML)

O que a questão diz: "Os comandos UPDATE, DROP e DELETE constituem a Linguagem de Manipulação de Dados (DML)."

O Erro (O Quê): Categorizar a instrução de destruição estrutural (DROP) como manipulação de dados inseridos (DML).

A Regra (Por Quê): UPDATE e DELETE manipulam as "linhas" (dados), por isso são **DML** (Data Manipulation Language). O comando DROP não mexe com o dado e sim destrói a "estrutura da tabela ou do banco", pertencendo à classe **DDL** (Data Definition Language) ao lado de CREATE e ALTER.

Q6 (Saneamento - Data Cleaning)

O que a questão diz: "Em um pipeline de ETL da conta de clientes, os dados transferidos não podem sofrer tratamento em lote (bulk) utilizando a instrução INSERT INTO ... SELECT com REPLACE simultaneamente."

O Erro (O Quê): Negar que a instrução de sub-consulta pode carregar dados manipulados através de funções de string nativas do SGBD de forma atômica.

A Regra (Por Quê): O processo massivo de Saneamento no ETB-PAY usa `INSERT INTO tb_nova (cpf) SELECT REPLACE(REPLACE(cpf, '-', ''), '.', '') FROM tb_suja`. Esta sintaxe clássica e performática limpa o lixo sintático "durante o voo", movendo de uma fonte suja para um destino limpo usando substituição por strings vazias de forma impecável.

Q7 (Consultas com INNER JOIN)

O que a questão diz: "O código SQL adequado para alinhar dados de `tb_clientes` com `tb_contas` omite a indicação da Chave Estrangeira com a cláusula `WHERE` em bancos modernos, priorizando o uso simples do `CROSS JOIN`."

O Erro (O Quê): Recomendar o cruzamento desordenado `CROSS JOIN` (que gera um produto cartesiano catastrófico) no lugar da junção condicional `ON`.

A Regra (Por Quê): A regra áurea relacional exige o `INNER JOIN` ou as variantes de `OUTTER JOIN` atreladas imperativamente à cláusula `ON`. Exemplo cirúrgico: `... FROM tb_clientes INNER JOIN tb_contas ON tb_clientes.id = tb_contas.id_cliente`. A falha na condição de igualdade desmorona a performance e integridade dos resultados.

Q8 (A Rasteira do SGBD Embedded)

O que a questão diz: "O SQLite caracteriza-se por ser um SGBD embedded (embutido), funcionando como um poderoso serviço integrado em background que suporta múltiplas instâncias TCP simultâneas."

O Erro (O Quê): Afirmar que funciona como um serviço em background que provê conexão TCP (rede).

A Regra (Por Quê): A palavra "Embedded" em SQLite significa literalmente a inexistência da camada de serviço/daemon TCP em background (o chamado Serverless). O banco é manipulado como uma biblioteca (arquivo estático local) diretamente consumido pelo programa (ex: pela aplicação Java do ETB-PAY) sob chamadas no próprio sistema operacional (Direct Disk I/O).

Q9 (O Perigo dos Registros Órfãos)

O que a questão diz: "A exclusão de um registro na tabela principal, mesmo que efetuada sem a configuração de cláusulas como CASCADE ou RESTRICT, é garantida em compliance porque a tabela dependente não reterá o Foreign Key."

O Erro (O Quê): Afirmar absurdamente que a exclusão não reterá o ID sem uma configuração ativa (gerando compliance falso).

A Regra (Por Quê): Sem o ON DELETE CASCADE ou ON DELETE RESTRICT ativado, deletar um pai cria imediatamente um **Registro Órfão** na tabela filha (uma conta bancária atrelada a um cliente fantasma). Isso viola severamente o Compliance do BACEN ao arruinar o Princípio da Integridade Referencial, causando a corrupção lógica irremediável dos saldos e do banco.

SEÇÃO C: O CHECKLIST DO EDITAL OFICIAL

Mapeamento para Revisão Final (AV3)

- ✓ Fundamentos de SGBD (Client-Server vs. Embedded)
- ✓ Os 3 Níveis de Abstração de Dados
- ✓ Regras de Normalização Relacional (1FN, 2FN, 3FN)
- ✓ Comandos SQL: DDL, DML e DQL
- ✓ Álgebra Relacional: INNER JOIN
- ✓ Integridade Referencial e Comportamento CASCADE